

AI FAQ Assistant API

Project Demonstration Report

Project Demonstration Phase

Project Name	AI FAQ Assistant API
Team ID	SWTID-2026-1291
Date	30 September 2026

1. Demonstration Objective

The objective of the project demonstration is to demonstrate the working functionality of the AI FAQ Assistant API and show how the different backend modules operate together.

The demonstration covers user authentication, JWT-based protected access, role-based authorization, FAQ management, category management, feedback, Google Gemini AI integration, keyword-based FAQ search, and AI-assisted answer generation.

The current project is implemented as a backend API using Express.js and MongoDB. The demonstration therefore focuses on API requests, responses, middleware protection, database operations, and AI service integration.

2. Demonstration Environment

The demonstration is performed using:

- Visual Studio Code
- Node.js
- Express.js
- MongoDB Atlas
- Google Gemini AI
- Thunder Client / API client

The backend is accessed through:

`http://localhost:5000`

The project uses environment variables for database, JWT, and Gemini configuration. Confidential values are not included in this report.

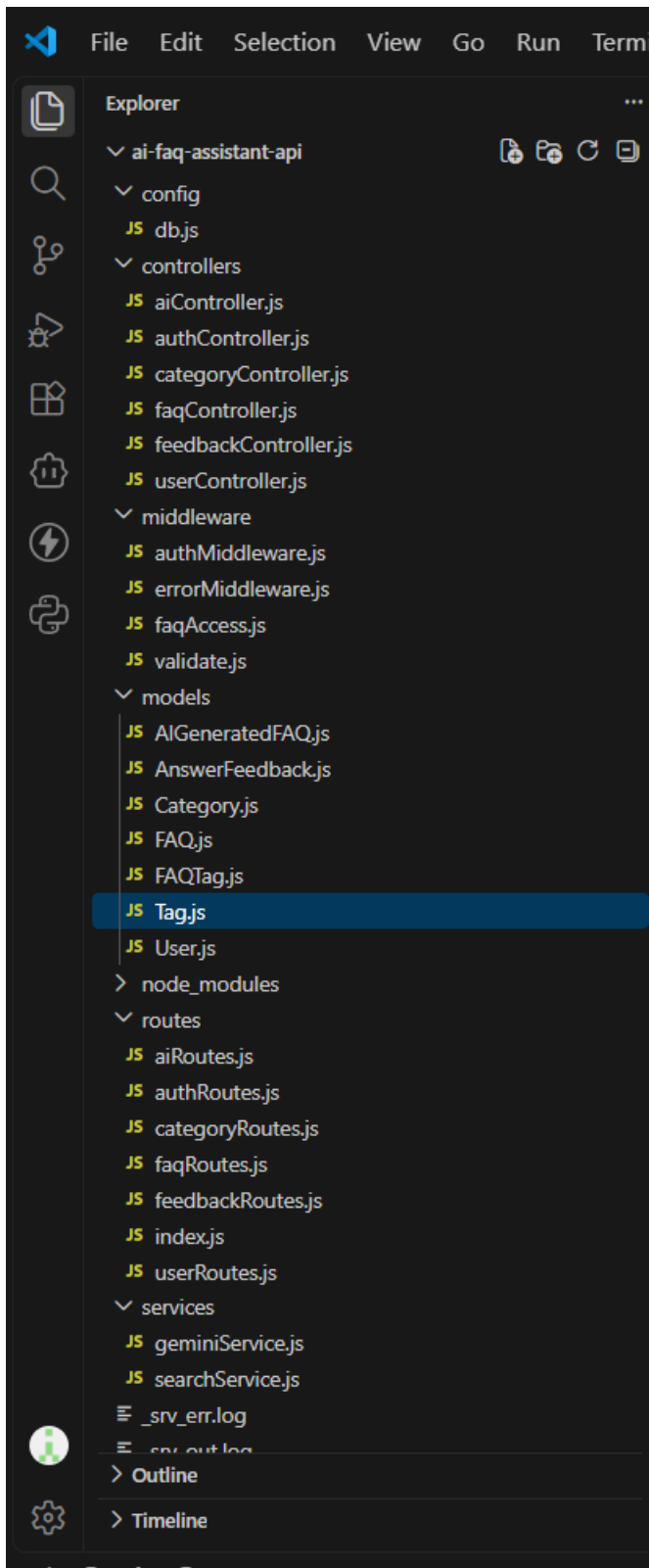


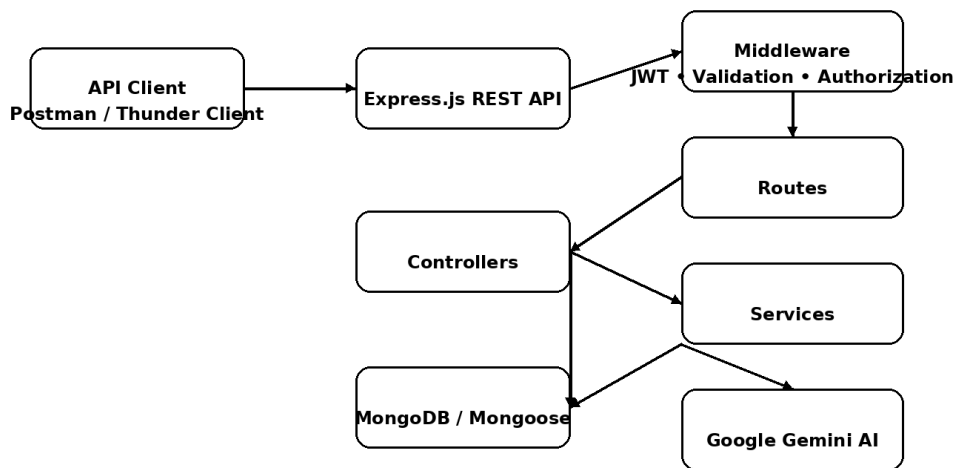
Figure 1. AI FAQ Assistant API project structure showing controllers, models, middleware, routes, services, and configuration files.

3. Demonstration Flow

The project is demonstrated in the following sequence:

- Start the Express.js backend and verify the health endpoint.
- Register and authenticate a user.
- Use the JWT token to access protected APIs.
- Demonstrate role-based access for admin and content creator operations.
- Create, retrieve, update, delete, and search FAQ records.
- Create and retrieve categories.
- Submit FAQ feedback.
- Generate an FAQ using Google Gemini.
- Save an AI-generated FAQ and request an AI-assisted answer.

AI FAQ Assistant API — System Architecture



Request flow: API Client → Express → Middleware → Routes → Controllers → Services/Models → MongoDB / Gemini

Figure 2. System architecture showing client requests, Express API, middleware, controllers, services, MongoDB, and Gemini AI integration.

4. Backend Startup

The backend server is started and the API is made available for testing. The application connects to MongoDB before starting the HTTP server.

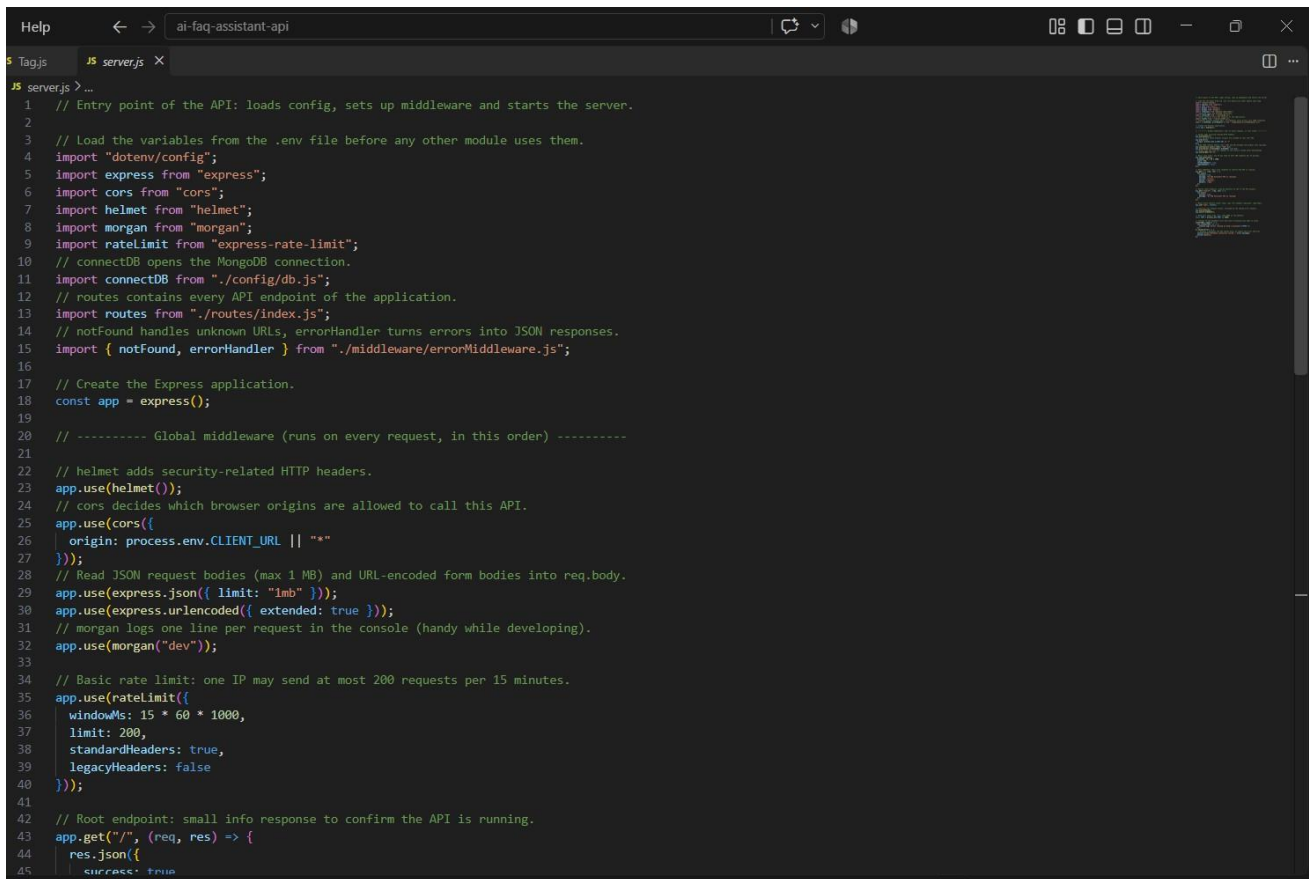
The health-check endpoint is used to verify that the application is running successfully.

GET /health

The root endpoint also provides API status information:

GET /

The project uses port 5000 by default.



```
1 // Entry point of the API: loads config, sets up middleware and starts the server.
2
3 // Load the variables from the .env file before any other module uses them.
4 import "dotenv/config";
5 import express from "express";
6 import cors from "cors";
7 import helmet from "helmet";
8 import morgan from "morgan";
9 import ratelimit from "express-rate-limit";
10 // connectDB opens the MongoDB connection.
11 import connectDB from "../config/db.js";
12 // routes contains every API endpoint of the application.
13 import routes from "../routes/index.js";
14 // notFound handles unknown URLs, errorHandler turns errors into JSON responses.
15 import { notFound, errorHandler } from "../middleware/errorMiddleware.js";
16
17 // Create the Express application.
18 const app = express();
19
20 // ----- Global middleware (runs on every request, in this order) -----
21
22 // helmet adds security-related HTTP headers.
23 app.use(helmet());
24 // cors decides which browser origins are allowed to call this API.
25 app.use(cors({
26   origin: process.env.CLIENT_URL || "*"
27 }));
28 // Read JSON request bodies (max 1 MB) and URL-encoded form bodies into req.body.
29 app.use(express.json({ limit: "1mb" }));
30 app.use(express.urlencoded({ extended: true }));
31 // morgan logs one line per request in the console (handy while developing).
32 app.use(morgan("dev"));
33
34 // Basic rate limit: one IP may send at most 200 requests per 15 minutes.
35 app.use(ratelimit({
36   windowMs: 15 * 60 * 1000,
37   limit: 200,
38   standardHeaders: true,
39   legacyHeaders: false
40 }));
41
42 // Root endpoint: small info response to confirm the API is running.
43 app.get("/", (req, res) => {
44   res.json({
45     success: true
```

Figure 3. server.js showing Express configuration, middleware, rate limiting, route mounting, health endpoint, and server startup.

5. Authentication Demonstration

The registration API is demonstrated first.

POST /api/auth/register

A new user is created using valid registration details. Passwords are stored securely using bcrypt hashing in the User model.

Next, the login API is demonstrated:

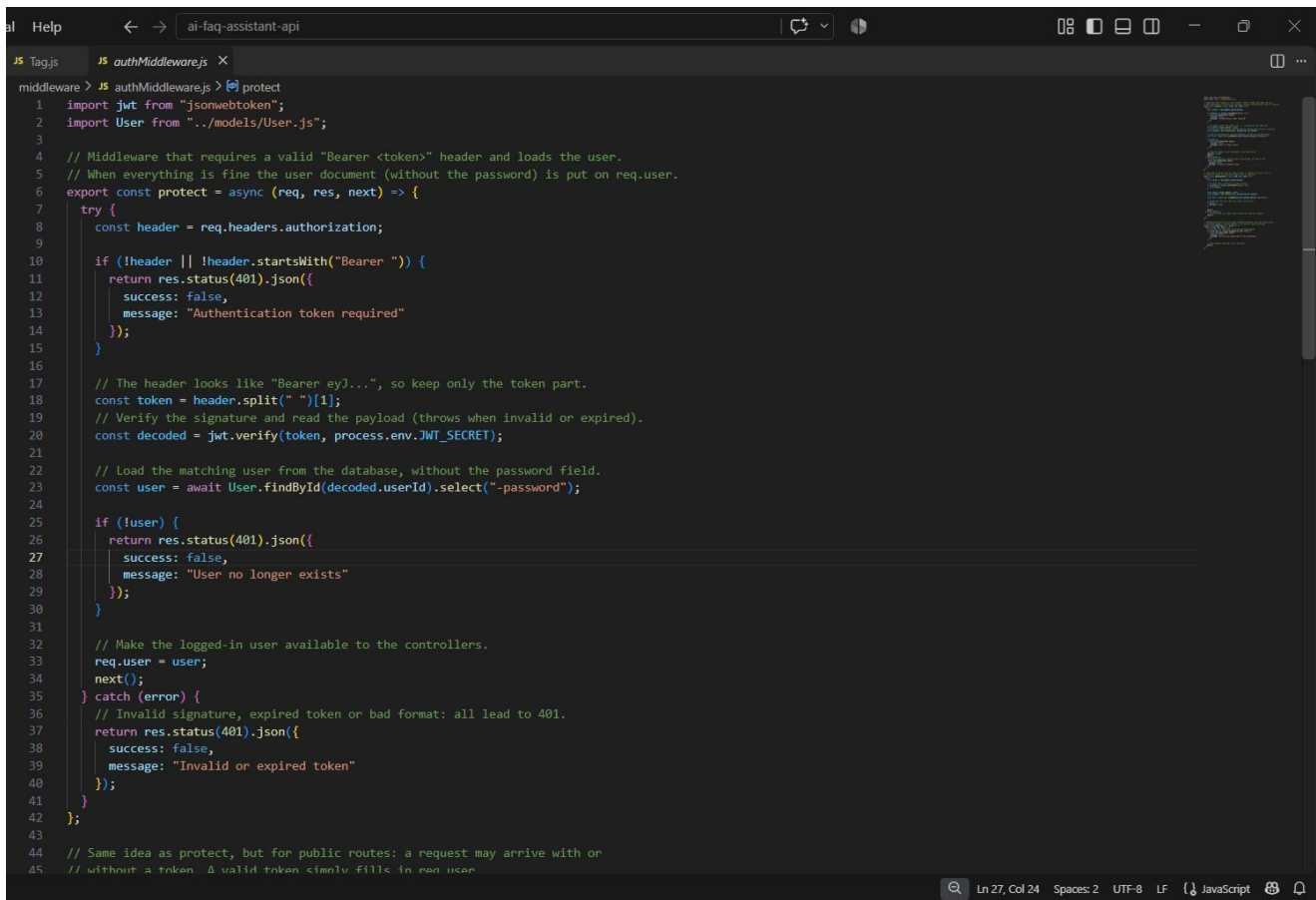
POST /api/auth/login

After successful authentication, a JWT token is returned. The token is then supplied as a Bearer token when accessing protected APIs.

The profile API confirms the authenticated user:

GET /api/auth/profile

Actual passwords and JWT tokens are intentionally excluded from this report.



```
middleware > JS authMiddleware.js > protect
1  import jwt from "jsonwebtoken";
2  import User from "../models/User.js";
3
4  // Middleware that requires a valid "Bearer <token>" header and loads the user.
5  // When everything is fine the user document (without the password) is put on req.user.
6  export const protect = async (req, res, next) => {
7    try {
8      const header = req.headers.authorization;
9
10     if (!header || !header.startsWith("Bearer ")) {
11       return res.status(401).json({
12         success: false,
13         message: "Authentication token required"
14       });
15     }
16
17     // The header looks like "Bearer eyJ..."., so keep only the token part.
18     const token = header.split(" ")[1];
19     // Verify the signature and read the payload (throws when invalid or expired).
20     const decoded = jwt.verify(token, process.env.JWT_SECRET);
21
22     // Load the matching user from the database, without the password field.
23     const user = await User.findById(decoded.userId).select("-password");
24
25     if (!user) {
26       return res.status(401).json({
27         success: false,
28         message: "User no longer exists"
29       });
30     }
31
32     // Make the logged-in user available to the controllers.
33     req.user = user;
34     next();
35   } catch (error) {
36     // Invalid signature, expired token or bad format: all lead to 401.
37     return res.status(401).json({
38       success: false,
39       message: "Invalid or expired token"
40     });
41   }
42 };
43
44 // Same idea as protect, but for public routes: a request may arrive with or
45 // without a token. A valid token simply fills in req.user
```

Figure 4. Authentication middleware and controller logic showing Bearer-token verification and protected user access.

6. Role-Based Access Control Demonstration

The project supports the following user roles:

- Admin
- Content Creator
- User

The demonstration shows that different users have different permissions. For example:

- An Admin can manage users, categories, and system-level resources.
- A Content Creator can create and manage FAQ content and use the AI FAQ generation features.
- A User can access public FAQ content, search FAQs, submit feedback, and use the protected AI answer feature.

The implementation uses authentication middleware, role authorization, and FAQ ownership protection.

authMiddleware.js — JWT Authentication and Authori

```
1 import jwt from "jsonwebtoken";
2 import User from "../models/User.js";
3
4 export const protect = async (req, res, next) => {
5   try {
6     const header = req.headers.authorization;
7
8     if (!header || !header.startsWith("Bearer ")) {
9       return res.status(401).json({
10         success: false,
11         message: "Authentication token required"
12       });
13     }
14
15     const token = header.split(" ")[1];
16     const decoded = jwt.verify(token, process.env.JWT_SECRET);
17
18     const user = await User.findById(decoded.userId).select("-password");
19
20     if (!user) {
21       return res.status(401).json({
22         success: false,
23         message: "User no longer exists"
24       });
25     }
26
27     req.user = user;
28     next();
29   } catch (error) {
30     return res.status(401).json({
31       success: false,
32       message: "Invalid or expired token"
33     });
34   }
35 };
36
37 export const optionalAuth = async (req, res, next) => {
38   try {
39     const header = req.headers.authorization;
40
41     if (!header || !header.startsWith("Bearer ")) {
42       return next();
43     }
44
45     const token = header.split(" ")[1];
46     const decoded = jwt.verify(token, process.env.JWT_SECRET);
47
48     const user = await User.findById(decoded.userId).select("-password");
49
50     if (user) {
51       req.user = user;
52     }
53
54     next();
55   } catch (error) {
56     // Invalid token on a public route should not block the request.
57     next();
58   }
59 };
60
61 export const authorize = (...roles) => {
62   return (req, res, next) => {
63     if (!req.user || !roles.includes(req.user.role)) {
64       return res.status(403).json({
65         success: false,
66         message: "You are not authorized for this operation"
67       });
68     }
69
70     next();
71   };
72 }
```

Figure 5. Authentication and authorization middleware used before protected operations.

7. FAQ Management Demonstration

The FAQ management APIs are demonstrated using an API client.

POST /api/faqs

GET /api/faqs

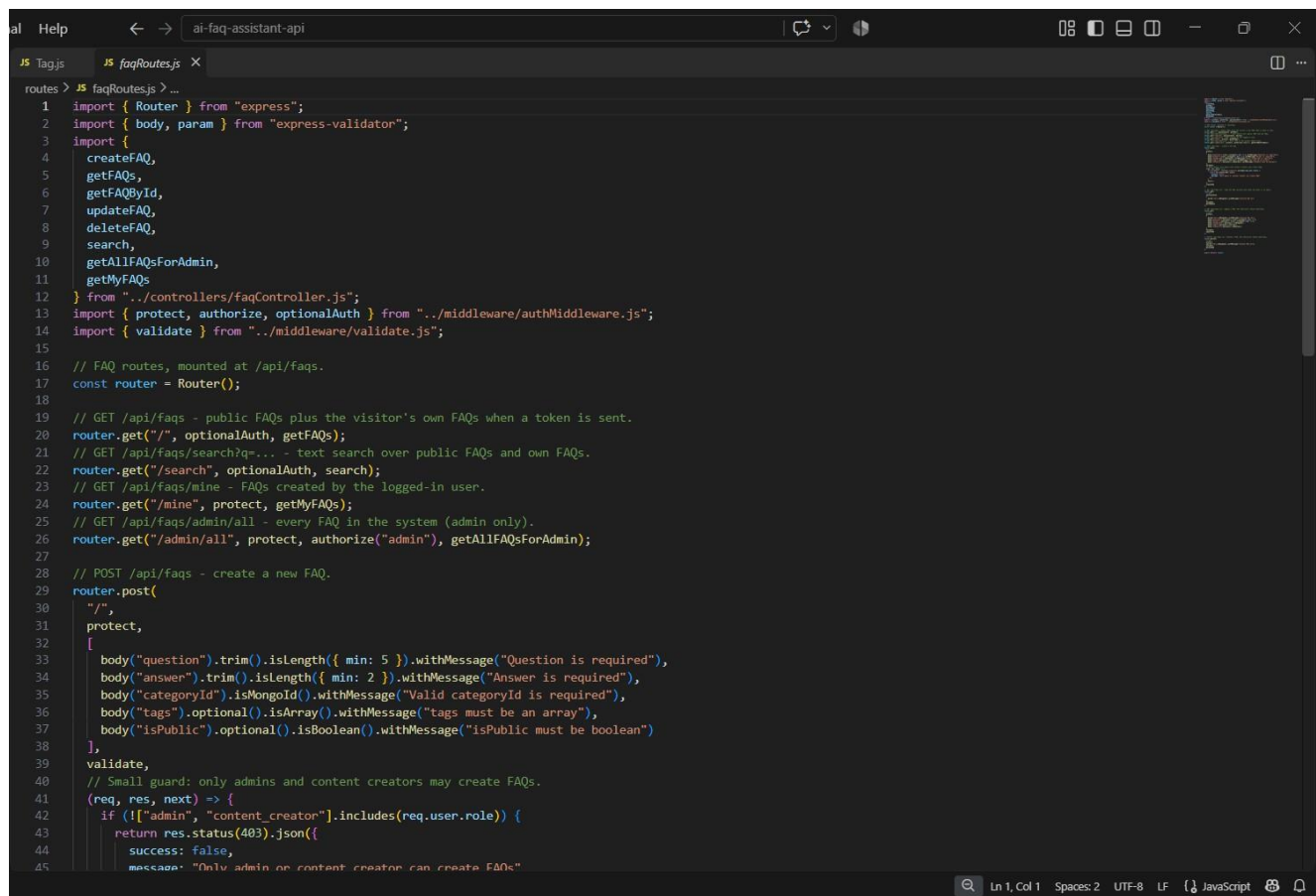
GET /api/faqs/:id

GET /api/faqs/mine

PUT /api/faqs/:id

DELETE /api/faqs/:id

The demonstration shows FAQ creation, retrieval, modification, deletion, ownership protection, visibility control, and view counting. Administrators can manage all FAQ records, while content creators are protected by ownership rules for their own FAQ records.



```
1 import { Router } from "express";
2 import { body, param } from "express-validator";
3 import {
4   createFAQ,
5   getFAQs,
6   getFAQById,
7   updateFAQ,
8   deleteFAQ,
9   search,
10  getAllFAQsForAdmin,
11  getMyFAQs
12 } from "../controllers/faqController.js";
13 import { protect, authorize, optionalAuth } from "../middleware/authMiddleware.js";
14 import { validate } from "../middleware/validate.js";
15
16 // FAQ routes, mounted at /api/faqs.
17 const router = Router();
18
19 // GET /api/faqs - public FAQs plus the visitor's own FAQs when a token is sent.
20 router.get("/", optionalAuth, getFAQs);
21 // GET /api/faqs/search?q=... - text search over public FAQs and own FAQs.
22 router.get("/search", optionalAuth, search);
23 // GET /api/faqs/mine - FAQs created by the logged-in user.
24 router.get("/mine", protect, getMyFAQs);
25 // GET /api/faqs/admin/all - every FAQ in the system (admin only).
26 router.get("/admin/all", protect, authorize("admin"), getAllFAQsForAdmin);
27
28 // POST /api/faqs - create a new FAQ.
29 router.post(
30   "/",
31   protect,
32   [
33     body("question").trim().isLength({ min: 5 }).withMessage("Question is required"),
34     body("answer").trim().isLength({ min: 2 }).withMessage("Answer is required"),
35     body("categoryId").isMongoId().withMessage("Valid categoryId is required"),
36     body("tags").optional().isArray().withMessage("tags must be an array"),
37     body("isPublic").optional().isBoolean().withMessage("isPublic must be boolean")
38   ],
39   validate,
40   // Small guard: only admins and content creators may create FAQs.
41   (req, res, next) => {
42     if (!["admin", "content_creator"].includes(req.user.role)) {
43       return res.status(403).json({
44         success: false,
45         message: "Only admin or content creator can create FAQs"
46       });
47     }
48     next();
49   }
50 );
```

Figure 6. FAQ routes showing public retrieval, protected creation, ownership-controlled update/delete, and administrator operations.

8. Category and Feedback Demonstration

The category APIs are used to organize FAQ content.

GET /api/categories

POST /api/categories

PUT /api/categories/:id

DELETE /api/categories/:id

Category creation, update, and deletion are restricted to administrators.

The feedback API allows an authenticated user to submit feedback for an FAQ.

POST /api/feedback

Feedback contains the FAQ reference, a helpful/not-helpful value, and an optional comment. Administrators can retrieve feedback records.

GET /api/feedback

9. AI FAQ Generation Demonstration

Google Gemini is used to generate FAQ content from a user-provided prompt.

POST /api/ai/generate-faq

The generated result is structured around:

- question
- answer
- topicSuggestions

Generated FAQ records are stored in the AIGeneratedFAQ collection. Authorized administrators and content creators can review generated records and save suitable generated content as an FAQ.

GET /api/ai/generated

POST /api/ai/generated/:id/save


```

1 import { GoogleGenAI } from "@google/genai"; "genai": Unknown word.
2
3 // Google Gemini client, created once using the API key from .env.
4 const ai = new GoogleGenAI({
5   apiKey: process.env.GEMINI_API_KEY
6 });
7
8 // Model name taken from .env, with a default model as fallback.
9 const model = process.env.GEMINI_MODEL || "gemini-3.6-flash";
10
11 // Ask Gemini to create one FAQ: a question, an answer and topic suggestions.
12 export const generateFAQ = async (prompt) => {
13   // Without an API key nothing can be sent, so fail early.
14   if (!process.env.GEMINI_API_KEY) {
15     throw new Error("GEMINI_API_KEY is missing in .env");
16   }
17
18   // The instructions ask the model to reply with JSON only.
19   const response = await ai.models.generateContent({
20     model,
21     contents: `
22     You are an AI FAQ content assistant.
23
24     Create one useful FAQ from the user's topic.
25
26     Return ONLY valid JSON with this exact structure:
27     {
28       "question": "string",
29       "answer": "string",
30       "topicSuggestions": ["string", "string", "string"]
31     }
32
33     User topic:
34     ${prompt}
35   `,
36     config: {
37       responseMimeType: "application/json"
38     }
39   });
40
41   // Turn the JSON text returned by the model into a JavaScript object.
42   const data = JSON.parse(response.text);
43
44   // Check that the model really returned a question and an answer.
45   if (!data.question || !data.answer) {

```

Figure 7. Gemini service implementation showing structured FAQ generation and generated answer processing.

10. Prompt Engineering Demonstration

The Gemini service uses structured prompts to request FAQ content in a predictable format. The generation prompt asks the model to return question, answer, and topic suggestions as JSON.

The AI answer feature also uses the supplied question together with FAQ context. The prompt instructs the model to avoid inventing facts when the available context is insufficient.

POST /api/ai/answer

The AI service is isolated in the service layer so that controller logic remains separate from Gemini-specific processing.

geminiService.js — Google Gemini Integration

```
1 import { GoogleGenAI } from "@google/genai";
2
3 const ai = new GoogleGenAI({
4   apiKey: process.env.GEMINI_API_KEY
5 });
6
7 const model = process.env.GEMINI_MODEL || "gemini-3.6-flash";
8
9 export const generateFAQ = async (prompt) => {
10   if (!process.env.GEMINI_API_KEY) {
11     throw new Error("GEMINI_API_KEY is missing in .env");
12   }
13
14   const response = await ai.models.generateContent({
15     model,
16     contents: `
17 You are an AI FAQ content assistant.
18
19 Create one useful FAQ from the user's topic.
20
21 Return ONLY valid JSON with this exact structure:
22 {
23   "question": "string",
24   "answer": "string",
25   "topicSuggestions": ["string", "string", "string"]
26 }
27
28 User topic:
29 ${prompt}
30 `
31     ,
32     config: {
33       responseType: "application/json"
34     }
35   });
36
37   const data = JSON.parse(response.text);
38
39   if (!data.question || !data.answer) {
40     throw new Error("Gemini returned incomplete FAQ data");
41   }
42
43   return {
44     question: data.question,
45     answer: data.answer,
46     topicSuggestions: Array.isArray(data.topicSuggestions)
47       ? data.topicSuggestions
48       : []
49   };
50
51 export const generateAnswer = async (question, context = "") => {
52   if (!process.env.GEMINI_API_KEY) {
53     throw new Error("GEMINI_API_KEY is missing in .env");
54   }
55
56   const response = await ai.models.generateContent({
57     model,
58     contents: `
59 Answer the user's FAQ question clearly and briefly.
60
61 Use the provided FAQ context when it is relevant.
62 Do not invent facts when the context is insufficient.
63
64 Question:
65 ${question}
66
67 FAQ Context:
68 ${context}
69 `
70   });
71
72   return response.text;
73 };
```

Figure 8. Gemini service showing the structured prompt and JSON-oriented response handling.

11. Search Demonstration

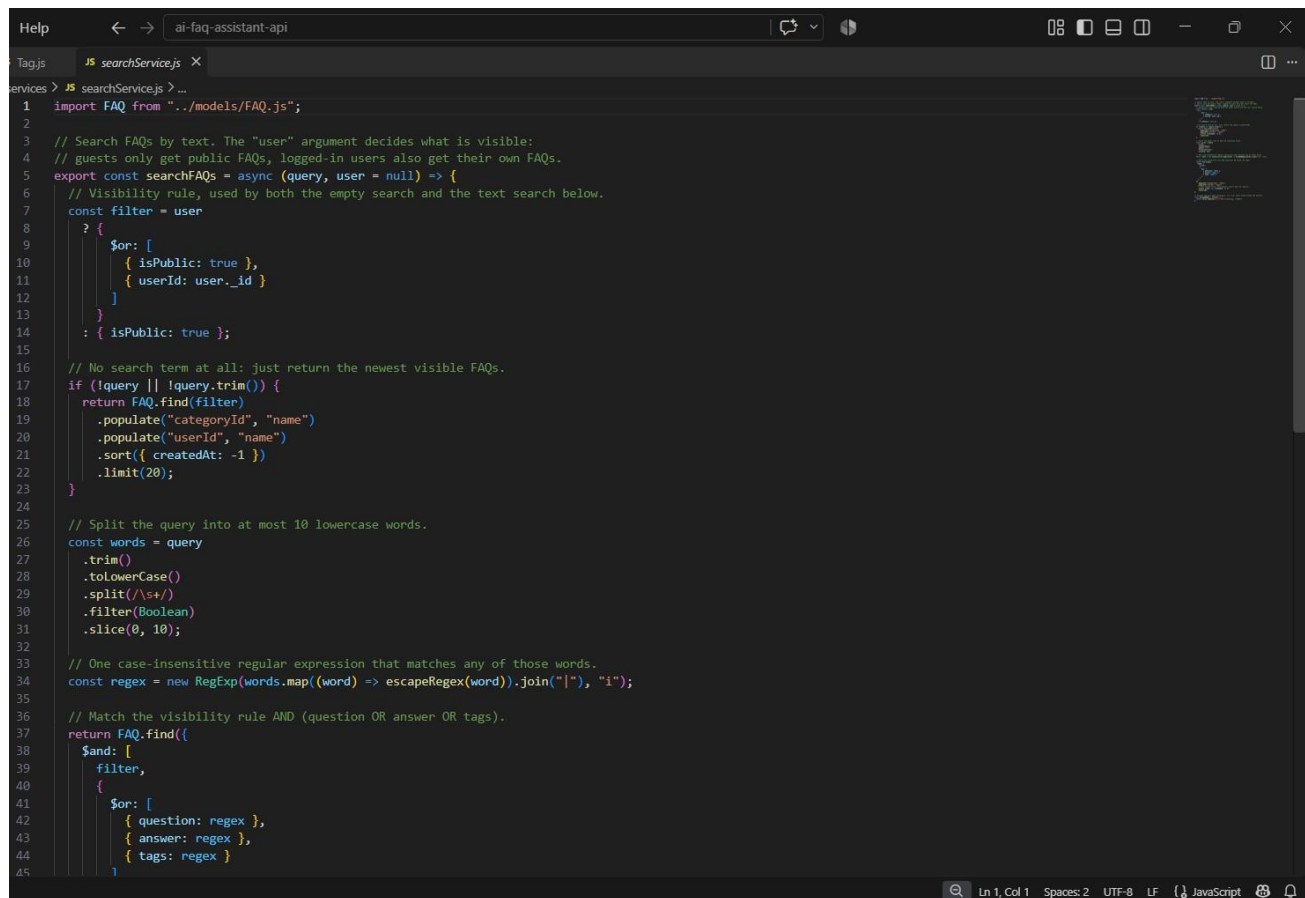
The current project provides keyword-based FAQ search. It does not implement MongoDB Atlas Search as a separate Atlas Search index.

```
GET /api/faqs/search?q=<search-term>
```

The search service:

- Restricts unauthenticated users to public FAQs.
- Allows authenticated users to access public FAQs and their own private FAQs.
- Searches question, answer, and tags using escaped keyword patterns.
- Returns populated category and user information.
- Sorts results using views and creation time and limits the result set.

The FAQ model also defines a MongoDB text index over question, answer, and tags. The current README explicitly describes the implemented search as keyword-based rather than vector search.



```
1 import FAQ from "../models/FAQ.js";
2
3 // Search FAQs by text. The "user" argument decides what is visible:
4 // guests only get public FAQs, logged-in users also get their own FAQs.
5 export const searchFAQs = async (query, user = null) => {
6   // Visibility rule, used by both the empty search and the text search below.
7   const filter = user
8     ? {
9       $or: [
10        { isPublic: true },
11        { userId: user._id }
12      ]
13     }
14     : { isPublic: true };
15
16   // No search term at all: just return the newest visible FAQs.
17   if (!query || !query.trim()) {
18     return FAQ.find(filter)
19       .populate("categoryId", "name")
20       .populate("userId", "name")
21       .sort({ createdAt: -1 })
22       .limit(20);
23   }
24
25   // Split the query into at most 10 lowercase words.
26   const words = query
27     .trim()
28     .toLowerCase()
29     .split(/\s+/)
30     .filter(Boolean)
31     .slice(0, 10);
32
33   // One case-insensitive regular expression that matches any of those words.
34   const regex = new RegExp(words.map(word => escapeRegex(word)).join("|"), "i");
35
36   // Match the visibility rule AND (question OR answer OR tags).
37   return FAQ.find({
38     $and: [
39       filter,
40       {
41         $or: [
42           { question: regex },
43           { answer: regex },
44           { tags: regex }
45         ]
46       }
47     ]
48   });
49 }
```

Figure 9. Search service implementation showing public/private FAQ visibility rules and keyword matching across question, answer, and tags.

12. AI-Assisted Answer Demonstration

The project includes an AI-assisted answer endpoint that combines FAQ context with Google Gemini.

The controller first obtains relevant public FAQ context and then passes the question and context to the Gemini service. The generated response is returned to the authenticated user.

MongoDB Atlas Vector Search and embedding-based semantic search are not implemented in the current project source. Therefore, no Vector Search demonstration is claimed in this report.

AI FAQ Assistant API — Entity Relationship Diagram

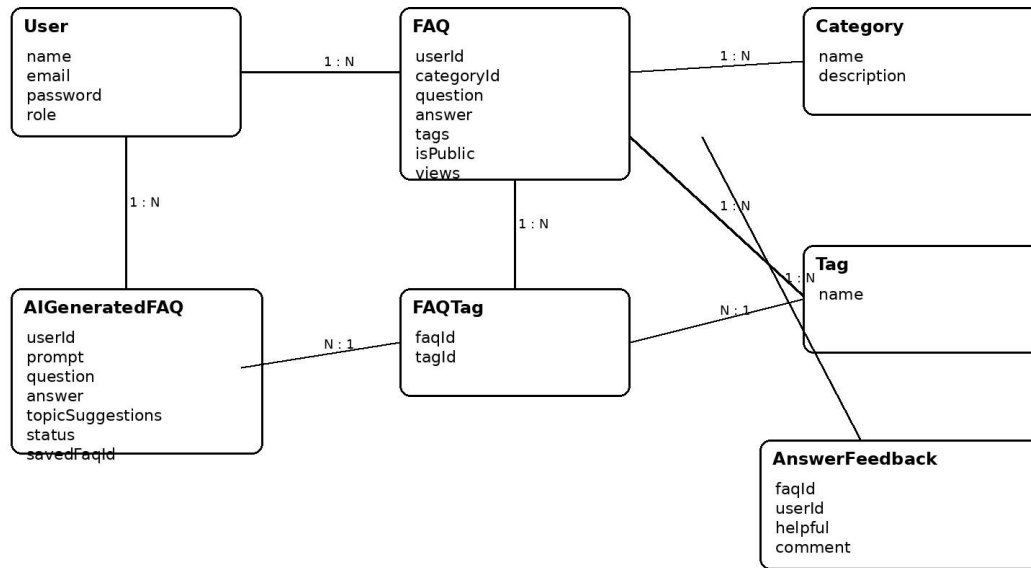


Figure 10. Database relationship diagram for users, FAQs, categories, tags, AI-generated FAQs, and feedback used by the current project.

13. Demonstration Outcome

The demonstration verifies the integrated operation of the AI FAQ Assistant backend. The following functionality is represented by the current implementation:

- User registration and login
- JWT-based protected access
- Role-based authorization
- FAQ management and ownership protection
- Category management
- FAQ feedback
- Google Gemini FAQ generation
- AI-assisted answer generation
- Keyword-based FAQ search
- MongoDB data storage and relationships

The API testing guide supplied with the project defines a broader sequence of endpoint checks. This report does not claim a fabricated pass/fail count; only results that are actually demonstrated or documented are described.

Confidential credentials, JWT tokens, API keys, and database connection strings are excluded from the demonstration report.

Conclusion

The AI FAQ Assistant API demonstrates a modular backend system that combines secure authentication, role-based authorization, FAQ and category management, feedback handling, keyword-based search, and Google Gemini AI services within an Express.js and MongoDB application.

The implementation separates routes, controllers, middleware, models, and services, making the major backend responsibilities easier to test and maintain.

The current demonstration reflects the features present in the project source. MongoDB Atlas Search and MongoDB Atlas Vector Search are not presented as implemented features because they are not part of the current project version.